

Progettazione Space API

Monitoraggio Voyager 1 e Servizi NASA

Emanuele Polini, Manuel Puccioni, Dario Bani

Aprile 2026

Indice

1	Panoramica Generale	4
1.1	Tecnologie utilizzate	4
2	Architettura del Sistema	4
2.1	Flusso delle richieste	4
3	Struttura del Progetto	5
4	Configurazione REST	5
4.1	ApplicationConfig	5
5	Resource REST	5
5.1	VoyagerResource	5
5.1.1	GET /api/voyager/status	5
5.2	ISSResource	6
5.2.1	GET /api/iss/location	6
5.3	ApodResource	6
5.3.1	GET /api/apod/today	6
6	Service Applicativi	6
6.1	VoyagerService	6
6.2	ISSService	6
6.3	ApodService	6
7	Client HTTP	6
7.1	NasaClient	7
7.2	OpenNotifyClient	7
8	Model	7
8.1	VoyagerData	7
8.2	ISSLocation	7
8.3	ApodData	7

9	Utility	7
9.1	UnitConverter	7
9.2	TimeUtils	8
10	Formato delle Risposte	8
11	Gestione delle Richieste HTTP	8
12	Flusso di Interazione	8
13	Conclusioni	9

1 Panoramica Generale

Il progetto consiste nello sviluppo di una Web API REST realizzata in Java tramite JAX-RS e Jetty. L'applicazione ha lo scopo di fornire dati relativi allo spazio e alle missioni NASA, fungendo da intermediario tra il client e servizi esterni.

L'obiettivo principale del sistema è il monitoraggio della sonda Voyager 1, attraverso endpoint REST che restituiscono informazioni elaborate in formato JSON.

Il server è in grado di:

- ricevere richieste HTTP da client esterni;
- interrogare API pubbliche NASA;
- elaborare e convertire i dati ricevuti;
- restituire risposte JSON strutturate.

L'applicazione è sviluppata come server REST standalone utilizzando Jetty come web server embedded.

1.1 Tecnologie utilizzate

Tecnologia	Scopo
Java	Linguaggio principale
JAX-RS	Gestione API REST
Jetty	Web Server Embedded
Maven	Gestione dipendenze
JSON	Formato dati
NASA APIs	Recupero dati spaziali
Open Notify API	Posizione ISS

2 Architettura del Sistema

Il sistema segue una struttura suddivisa in livelli per separare le responsabilità applicative.

2.1 Flusso delle richieste

Client → Resource → Service → API Esterne
API Esterne → Service → JSON → Client

Quando un client effettua una richiesta HTTP, Jetty intercetta la richiesta e la inoltra a JAX-RS. Successivamente il sistema individua la Resource corretta, esegue la logica applicativa tramite i Service e restituisce una risposta JSON.

3 Struttura del Progetto

Il progetto è organizzato nei seguenti package:

Package	Responsabilità
resource	Gestione endpoint REST
service	Logica applicativa
model	Strutture dati JSON
client	Comunicazione con API esterne
utils	Conversioni e funzioni di supporto
config	Configurazione JAX-RS

4 Configurazione REST

4.1 ApplicationConfig

La classe ApplicationConfig definisce il punto di ingresso delle API REST.

Attraverso l'annotazione:

```
@ApplicationPath("/api")
```

viene definito il prefisso comune di tutti gli endpoint.

Esempio:

```
/api/voyager/status
```

```
/api/iss/location
```

5 Resource REST

Le Resource rappresentano gli endpoint accessibili dal client.

5.1 VoyagerResource

Gestisce le richieste relative alla sonda Voyager 1.

Endpoint disponibili:

5.1.1 GET /api/voyager/status

Restituisce informazioni sulla sonda Voyager 1.

Risposta 200 OK:

```
{
  "name": "Voyager 1",
  "distanceAu": 160.5,
  "speedKmS": 17,
  "yearsInSpace": 49
}
```

5.2 ISSResource

Gestisce le richieste relative alla posizione della Stazione Spaziale Internazionale.

5.2.1 GET /api/iss/location

Restituisce latitudine e longitudine aggiornate della ISS.

5.3 ApodResource

Gestisce i contenuti Astronomy Picture of the Day forniti dalla NASA.

5.3.1 GET /api/apod/today

Restituisce immagine, titolo e descrizione del giorno.

6 Service Applicativi

I Service contengono la logica principale dell'applicazione.

6.1 VoyagerService

La classe VoyagerService:

- recupera dati dalle API NASA;
- effettua elaborazioni;
- converte unità di misura;
- prepara i dati da restituire.

Attualmente il sistema utilizza dati simulati per Voyager 1 integrati con chiamate reali alle NASA APIs.

6.2 ISSService

Interroga la Open Notify API per ottenere la posizione corrente della ISS.

6.3 ApodService

Recupera dati relativi all'Astronomy Picture of the Day.

7 Client HTTP

Le classi client si occupano esclusivamente della comunicazione con API esterne.

7.1 NasaClient

Effettua richieste HTTP verso le NASA APIs.

Esempio endpoint utilizzato:

`https://api.nasa.gov/planetary/apod`

7.2 OpenNotifyClient

Effettua richieste HTTP verso la Open Notify API per ottenere i dati della ISS.

8 Model

Le classi Model rappresentano i dati restituiti sotto forma di JSON.

8.1 VoyagerData

Contiene:

- nome della sonda;
- distanza dalla Terra;
- velocità;
- anni trascorsi dal lancio.

8.2 ISSLocation

Contiene:

- latitudine;
- longitudine.

8.3 ApodData

Contiene:

- titolo;
- descrizione;
- URL immagine.

9 Utility

9.1 UnitConverter

Contiene funzioni di conversione tra chilometri e unità astronomiche.

9.2 TimeUtils

Contiene funzioni per il calcolo del tempo trascorso dal lancio della Voyager 1.

10 Formato delle Risposte

Tutte le risposte vengono restituite in formato JSON.

Esempio:

```
{
  "name": "Voyager 1",
  "distanceAu": 160.5,
  "speedKmS": 17,
  "yearsInSpace": 49
}
```

11 Gestione delle Richieste HTTP

Il sistema supporta principalmente richieste HTTP di tipo GET per la lettura dei dati.

Le annotazioni JAX-RS utilizzate sono:

Annotazione	Funzione
@Path	Definisce l'URL
@GET	Gestisce richieste GET
@Produces	Definisce il formato di risposta

12 Flusso di Interazione

Il diagramma seguente rappresenta il flusso completo di una richiesta REST relativa al monitoraggio della sonda Voyager 1.

Il client effettua una richiesta HTTP verso il server Jetty. La richiesta viene intercettata dalla classe VoyagerResource che delega la logica applicativa al VoyagerService. Successivamente il service effettua una chiamata HTTP verso le NASA APIs, riceve i dati JSON, li elabora e restituisce una risposta JSON strutturata al client.

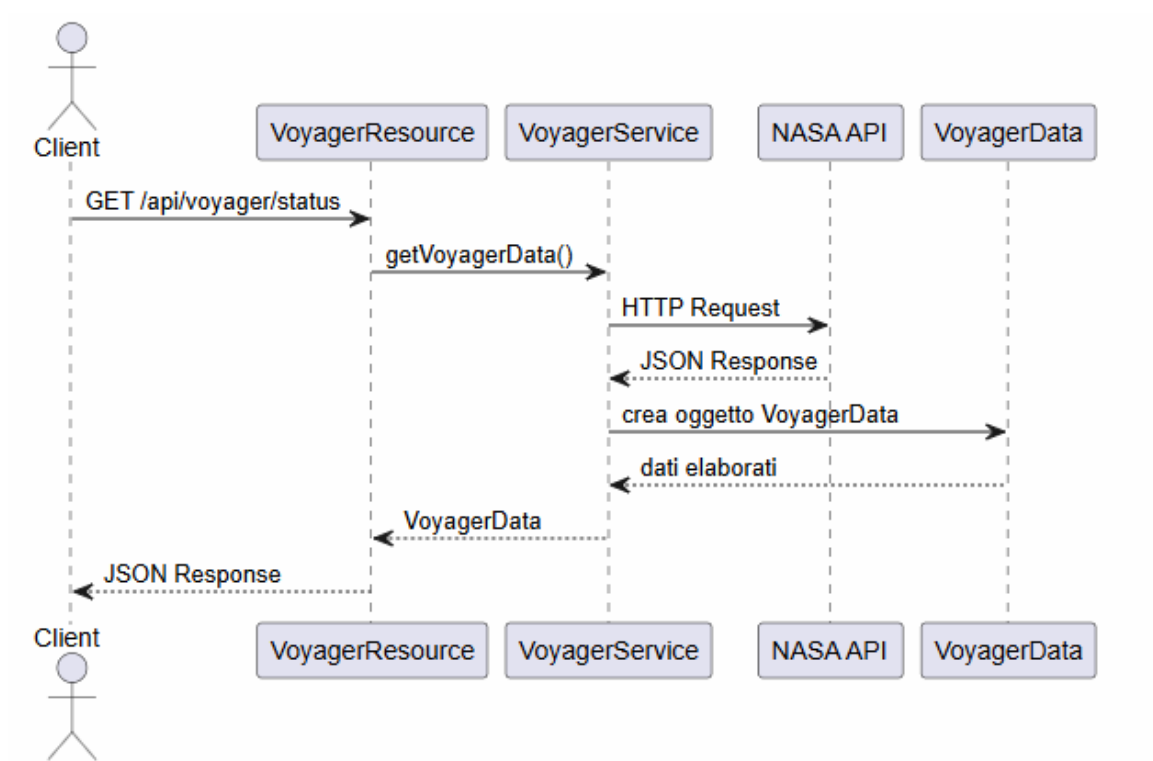


Figura 1: Sequence Diagram - Flusso richiesta Voyager 1

13 Conclusioni

Il progetto realizza una Web API REST modulare e scalabile in grado di integrare servizi spaziali esterni tramite HTTP.

La separazione tra Resource, Service, Client e Model permette di mantenere il codice organizzato e facilmente estendibile.

L'architettura adottata consente inoltre di aggiungere facilmente nuovi endpoint REST e nuovi servizi NASA in futuro.